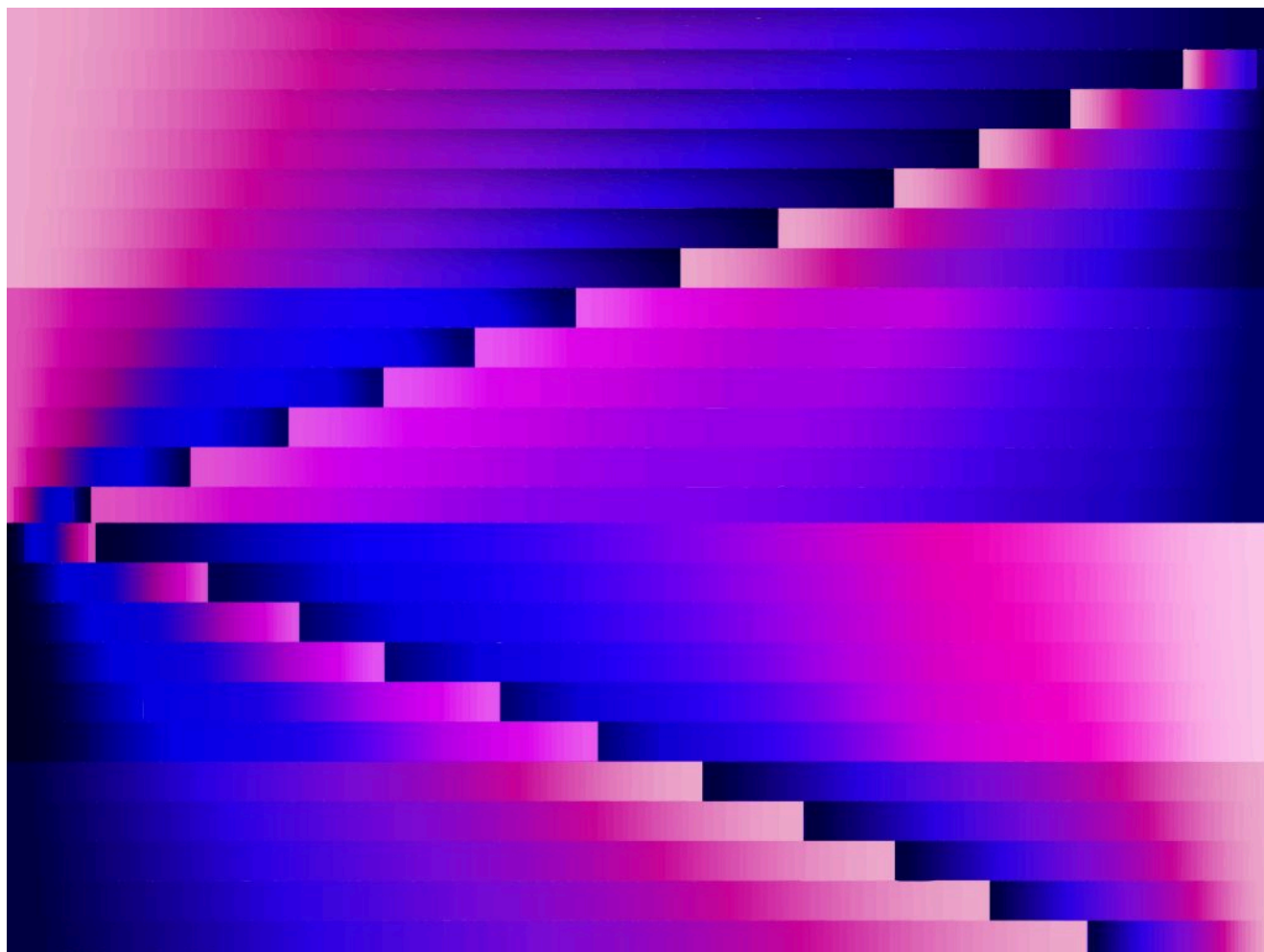


РАЗРАБОТКА ИИ / ИНСТРУМЕНТЫ ДЛЯ РАЗРАБОТЧИКОВ / РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Вашему инженерному подразделению нужен реестр ошибок, допущенных искусственным интеллектом

Не позволяйте инструментам для написания кода на основе ИИ масштабировать ошибки. Узнайте, как реестр ошибок ИИ и отдельная проверка обеспечивают соблюдение стандартов репозитория.

26 июня 2026 года, 10:00 [Анkit Джайн](#)



Найла Конита для Unsplash+

[Aviator](#) спонсировал этот пост.

Инструменты для написания кода на основе искусственного интеллекта не только помогают инженерам писать код быстрее. Они помогают инженерам

написанном ИИ, который явно ошибочен, а с кодом, который компилируется, проходит базовые проверки и выглядит правдоподобно, но содержит едва заметные ошибки, избыточен или не соответствует тому, что было нужно на самом деле.

На практике это обычно выглядит так: ИИ чрезмерно усложняет уровень абстракции для решения задачи, которая решается в 10 строк кода, игнорирует шаблоны, названия и архитектуру вашего репозитория, обращается к несуществующим API или копирует шаблоны, не понимая, зачем это нужно, например логику повторных попыток там, где она не нужна.

Подобные ошибки носят систематический характер, поэтому их можно предотвратить.

У вас есть CLAUDE.md и Skills, но...

Большинство команд пытаются дать ИИ более четкие инструкции. Они документируют свои стандарты в файле **CLAUDE.md**, настраивают Skills и описывают правила, которым должна следовать модель. Это правильный подход, но он не всегда работает.

Они просят тот же недетерминированный агент, который сгенерировал код, исправить собственные ошибки. Он может следовать написанным вами правилам. А может и не следовать. В любом случае нет никаких доказательств, никакого контрольного следа, и вы не можете заранее знать, какой результат получите. Файл CLAUDE.md — это входные данные для генерации. Это не система проверки.

«Файл CLAUDE.md — это входные данные для генерации. Это не система проверки».

Для надежной защиты от ошибок требуется нечто структурно обособленное: система, которая независимо проверяет выходные данные, использует другого агента и выдает один и тот же результат при работе с одним и тем же кодом.

Два уровня верификации

намерение. Вместо того чтобы рецензент читал diff и спрашивал: «Выглядит ли это правильно?», команда договаривается о том, что должен делать код, еще до его написания, а отдельная система верификации сверяет результат с этим соглашением.



Aviator — это платформа для повышения продуктивности разработчиков, которая помогает современным инженерным командам масштабировать код, сгенерированный искусственным интеллектом. Общий контекст, ускоренные проверки, детерминированная верификация и автоматизация процесса слияния и развертывания, созданная для работы с большим объемом кода, создаваемого ИИ.

Узнать больше →

ПОСЛЕДНЕЕ ОТ AVIATOR

Верификация — это не тестирование. Понимание разницы может спасти вашу кодовую базу

26 июня 2026 года

Сколько еще мы будем читать этот код?

23 июня 2026 года

Узкое место при проверке кода с помощью ИИ: почему чем быстрее генерируется код, тем медленнее его проверяют

3 июня 2026 года

УЗНАЙТЕ БОЛЬШЕ ОТ НАШЕГО СПОНСОРА

svo1975pvn1982@gmail.com

Отправить

Представьте, что это проверка здания. Здание не получает разрешение на в эксплуатацию от архитектора, который следит за тем, чтобы был забит каждый гвоздь. Разрешение выдает инспектор, который сверяет

агента — это строительство, конвейер верификатора выдает вердикты и доказательства по каждому критерию, а рецензент принимает решение на основе соответствия намерениям и качества доказательств.

«Вместо того чтобы рецензент читал diff и спрашивал: «Выглядит ли это правильно?», команда договаривается о том, что должен делать код, еще до того, как он будет написан».

Модель состоит из **двух слоев**, и понимание того, почему их два, а не один, — ключ к ее работе.

ТРЕНДОВЫЕ ИСТОРИИ

1. «Код должен генерироваться, а не поддерживаться»: Codeplain приводит аргументы в пользу разработки на основе спецификаций
2. Gemini CLI против. Antigravity: что работает, а что нет
3. Вашей команде разработчиков нужен реестр ошибок, допущенных искусственным интеллектом
4. «Пора навести порядок»: почему ИИ теперь проверяет код лучше, чем ваш коллега.
5. Циклы заменяют подсказки. Проверка скоро станет вашей самой большой проблемой.

Пользовательские критерии — это критерии приемки конкретного изменения, сгенерированные агентом на основе заявленного намерения или написанные вручную. Они применимы только к этому PR. Путь к конечной точке, форма ответа, поведение в случае сбоя и то, что явно выходит за рамки. Здесь содержится информация о намерении для конкретной задачи.

Критерии инвариантности берутся из каталога инвариантов команды и представляют собой правила, которые автоматически применяются к каждому соответствующему изменению. Если пользовательские критерии

вашем аккаунте и обновляются единожды для всех.

Ваши инварианты должны четко определять правило, но не содержать подробностей о реализации:

- Все обработчики HTTP должны вызывать промежуточное ПО для аутентификации перед выполнением любой бизнес-логики.
- «Все миграции должны содержать блок down».

Они определяются один раз и проверяются при каждом запуске.

Разработчикам не нужно включать их в спецификации, поскольку система автоматически загружает соответствующий набор.

Критерием превращения проверки в инвариант является повторяемость: все, что вы несколько раз публикуете в комментариях к обзору, должно стать инвариантом. Aviator делает это автоматически. Он автоматически создает инварианты на основе предыдущих комментариев.

При проверке оба уровня объединяются в единый список критериев приемки и проходят через один и тот же конвейер. Спецификация, добавляющая конечную точку статуса подписки, может содержать следующие пользовательские критерии:

Добавить конечную точку для проверки статуса подписки

Критерии приемки

– [] Конечная точка: GET /api/v1/subscription/status

– [] Ответ включает: status, renewal_date

Затем каталог инвариантов автоматически добавляет собственные критерии, например правило, согласно которому все HTTP-обработчики должны использовать AuthMiddleware. Проверка охватывает все эти критерии:

- ✓ Конечная точка существует по правильному пути (критерий пользователя)
- ✓ Ответ содержит статус и дату продления (критерий пользователя)

All must pass. The spec author didn't need to remember the authentication requirement. It was enforced by the catalog without anyone asking for it.

Invariants as the anti-slop registry

Invariants are what we call the 'anti-AI slop registry,' and that makes this work at scale. They address the most common category of AI slop: convention blindness, deprecated APIs, module boundaries the model doesn't know about, and security baselines that should apply everywhere. None of these are in the **model's training data** for your specific codebase. They live in the heads of your senior engineers and show up as recurring review comments.

Most invariants worth writing start as a review comment that's been left more than twice. Here is an example of turning a real review comment into an invariant:

Comment on PR #4173:

"Please don't write to users directly — go through

`UserRepository.UpdateProfile`. We had a partial-write bug last quarter from a similar pattern."

Invariant body:

```
1 | Copy
2 | Writes to the users table must go through UserRepository. Di
3 | UPDATE, or DELETE statements against the users table are not
4 | outside the repository package. Schema migrations under src,
```

Conditions: `file_path_glob: src/**/*.go` (skip non-Go files).

Category: `functional_correctness`.

You can mine historical review comments, cluster them, and generate invariant candidates for human approval. Each invariant you codify is a check that will never cost a reviewer time again.

I may have said that code review is a historical approval gate that no longer matches the shape of engineering work or that we can **stop reading the code**, but it will not happen overnight. In practice, over time, we move the human judgment

not 500-line diffs.

The other thing that sets this apart from a rules file is what happens at the time of **verification**. The writing agent and the verifying agent are different. They don't share context, they don't share blind spots, and the verifier produces a structured report per criterion — file references, reasoning, pass/fail/partial — not a gut-check opinion from the same model that wrote the code.

What we built, and what it found

At Aviator, we recently ran an experiment to test **the intent-driven verification approach**: what if the review happens before the code is written?

Instead of **AI writing code** and engineers reviewing it, the team spent time writing and reviewing scope, acceptance criteria, and edge cases before any implementation started. Then we handed it to an AI agent and let it build.

The result was about 6,000 lines of code. A second agent then verified the output against the 65 user criteria items in the spec. It took six minutes. 60 passed, 4 failed, and 1 was partial.

“You’re not building software anymore. You’re building the machine that builds software, and quality control is part of that machine.”

Human reviewers still found things, but design-level decisions were verified before any code was generated, and org invariants were enforced automatically throughout.

Instead of leaving the same comment for the fifteenth time, you’re identifying the pattern, writing it once, and letting the system enforce it on every change that follows. You’re not building software anymore. You’re building the machine that builds software, and **quality control is part of that machine**.



community of senior DevOps and senior software engineers focused on...

[Read more from Ankit Jain →](#)

SHARE THIS STORY



TRENDING STORIES

1. "Code should be regenerated, not maintained": Codeplain makes the case for spec-driven development
2. Gemini CLI vs. Antigravity: What works, not the spec sheet
3. Your engineering org needs an AI slop registry
4. "Time to clean up human slop": Why AI now reviews code better than your teammate.
5. Loops are replacing prompts. Verification is about to be your biggest problem.

INSIGHTS FROM OUR SPONSOR



Aviator is a developer productivity platform used by modern engineering teams to ship AI-generated code at scale. Shared context, faster reviews, deterministic verification, and merge-to-deploy automation built for the volume AI creates.

[Learn More →](#)

**Verification Is Not Testing. Understanding the Difference
Could Save Your Codebase**

June 2026

The AI Code Verification Bottleneck: Why Faster Code Generation Means Slower Reviews

3 June 2026

AI Code Review Is Still a Review

28 May 2026

Do we even need a better GitHub?

6 May 2026

Building Reusable AI Workflows: Templates for Common Engineering Tasks

22 April 2026

TNS DAILY NEWSLETTER

Receive a free roundup of the
most recent TNS articles in
your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

FOLLOW TNS



ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)